LABORATORIO N°3

"Aritmética de punto fijo y algoritmo exponencial en Assembly LC-3"

FACULTAD DE INGENIERÍA - UNAM

CARRERA:

Ingeniería en Computación

MATERIA:

IC221 - Arquitectura de computadoras

DOCENTES:

Dr. Ing. Roberto E. Carballo

Dr. Ing. Aldo J. Benitez.

Ing. Roberto N. Schuster

Ing. Patricia V. Feltan

Ing. Gustavo O. Rossler

Ing. Eduardo Díaz

Sr. Joaquín Nahuel Giménez

Sr. Joaquín Zuk

ALUMNOS:

Diaz, Agustina

Feltan, Angel Shaquel

Pires, Facundo Nahuel

GRUPO 6

1. ÍNDICE

Contenido

2. DESARROLLO 3
2.1. LABORATORIO 3A 3
2.2. LABORATORIO 3B 10
3. BITÁCORA DE DESARROLLO 31
Fecha: 7 y 14 de octubre de 2026 31
Fecha: 21 y 28 de octubre de 2026 31
Fecha: 4 de noviembre de 2026 Aprobación y organización 32
Fecha: 5 de noviembre de 2026 Presentación del LABORATORIO 3 32
4. CONCLUSIONES 32

2. DESARROLLO

2.1. LABORATORIO 3A

2.1.1. Lenguaje de alto nivel en C

lab3_punto_fijo.c | Líneas 1–39

```c
#include <stdio.h>
#include <stdint.h> // Necesaria para usar int16_t (entero de 16 bits)
#include <inttypes.h>

// Corrimiento aritmético a la derecha, también para valores negativos.
static int32_t corrimiento_derecha(int32_t valor, unsigned bits) {
    if (valor >= 0) {
        return valor >> bits;
    }
    return -1 - ((-1 - valor) >> bits);
}

// Verifica el resultado antes de convertirlo al entero de 16 bits del formato Q.
static void mostrar_resultado(const char *operacion, int32_t valor, int32_t escala) {
    if (valor < INT16_MIN || valor > INT16_MAX) {
        printf("%s: fuera de rango para el formato de salida.\n", operacion);
        return;
    }
    int16_t resultado = (int16_t)valor;
    printf("%s: %d -> Real: %g\n", operacion, (int)resultado, (double)resultado / escala);
}

int main(void) {
    // --- PUNTO 1: Declaración de variables e ingreso de datos en punto flotante ---
    float numero1;
    float numero2;

    // Ingreso del primer número (recomendado entre -8 y 7.99 para no desbordar Q3.12)
    printf("Ingrese el primer numero (use punto decimal): ");
    while (scanf("%f", &numero1) != 1) {
        if (feof(stdin) || ferror(stdin)) {
            return 1;
        }
        printf("Error: Entrada no valida. Ingrese un numero con punto decimal: ");
        int caracter;
        while ((caracter = getchar()) != '\n' && caracter != EOF) {
        }
    }

```

Figura 1

Figura 1: Funciones auxiliares e ingreso del primer operando (líneas 1 a 39).

lab3_punto_fijo.c | Líneas 40–72

```
40      // Ingreso del segundo número (recomendado entre -128 y 127.99 para no desbordar Q7.8)
41      printf("Ingrese el segundo numero (use punto decimal): ");
42      while (scanf("%f", &numero2) != 1) {
43          if (feof(stdin) || ferror(stdin)) {
44              return 1;
45          }
46          printf("Error: Entrada no valida. Ingrese un numero con punto decimal: ");
47          int caracter;
48          while ((caracter = getchar()) != '\n' && caracter != EOF) {
49          }
50      }
51
52      if (!(numero1 >= -8.0f && numero1 <= 32767.0f / 4096.0f) ||
53          !(numero2 >= -128.0f && numero2 <= 32767.0f / 256.0f)) {
54          puts("Error: Un numero esta fuera del rango de su formato Q.");
55          return 1;
56      }
57
58      // --- PUNTO 2: Conversión a Formato Q (Punto Fijo) ---
59
60      // Convertimos numero1 a Q3.12
61      // Multiplicamos por 4096 (que es 2^12) y hacemos casting a int16_t
62      int16_t num1_Q3_12 = (int16_t)(numero1 * 4096.0f);
63
64      // Convertimos numero2 a Q7.8
65      // Multiplicamos por 256 (que es 2^8) y hacemos casting a int16_t
66      int16_t num2_Q7_8 = (int16_t)(numero2 * 256.0f);
67
68      // Mostramos los resultados
69      printf("\n--- Resultados de la conversion ---\n");
70      printf("Numero 1 original: %g -> Guardado en Q3.12 como: %d\n", numero1, num1_Q3_12);
71      printf("Numero 2 original: %g -> Guardado en Q7.8 como: %d\n", numero2, num2_Q7_8);
72
```

Figura 2

Figura 2: Ingreso del segundo operando, validación y conversión a punto fijo (líneas 40 a 72).

lab3_punto_fijo.c | Líneas 73-99

```
73      // --- PUNTO 3: Compatibilización de formatos y Suma/Resta ---
74
75      // Convertimos num1 (Q3.12) a formato Q7.8
76      // Operación utilizada: Desplazamiento de bits a la derecha (>>) en 4 posiciones.
77      int32_t num1_convertido_a_Q7_8 = corrimiento_derecha(num1_Q3_12, 4);
78
79      // Ahora que ambos números están en Q7.8, se pueden sumar y restar de forma directa
80      int32_t suma_Q7_8 = num1_convertido_a_Q7_8 + num2_Q7_8;
81      int32_t resta_Q7_8 = num1_convertido_a_Q7_8 - num2_Q7_8;
82
83      printf("\n--- Punto 3: Suma y Resta (Formatos Compatibilizados) ---\n");
84      mostrar_resultado("Suma (en Q7.8)", suma_Q7_8, 256);
85      mostrar_resultado("Resta num1 - num2 (en Q7.8)", resta_Q7_8, 256);
86
87      // --- PUNTO 4: Multiplicación y Ajuste de Formato ---
88
89      // 1. Multiplicación cruda: Q3.12 * Q7.8 = Q10.20
90      // Usamos int32_t (32 bits) para evitar desborde al multiplicar.
91      int32_t mult_raw = (int32_t)num1_Q3_12 * (int32_t)num2_Q7_8;
92
93      // 2. Ajuste: Conservar en Q7.8. Desplazamos 12 posiciones a la derecha.
94      int32_t mult_Q7_8 = corrimiento_derecha(mult_raw, 12);
95
96      printf("\n--- Punto 4: Multiplicacion ---\n");
97      printf("Multiplicacion (cruda en Q10.20): %" PRId32 "\n", mult_raw);
98      mostrar_resultado("Multiplicacion ajustada a Q7.8", mult_Q7_8, 256);
99
```

Figura 3

Figura 3: Compatibilización de formatos, suma, resta y multiplicación (líneas 73 a 99).

lab3_punto_fijo.c | Líneas 100-117

```
100     // --- PUNTO 5: División conservando la mayor resolución posible ---
101
102     // Elegimos el formato Q3.12 para mantener 12 bits fraccionarios en el resultado.
103     // Sumamos 8 bits de ceros al numerador ANTES de dividir (desplazamiento a la izquierda).
104     // En C, multiplicar por 256 aplica el mismo factor sin desplazar un entero negativo.
105     int32_t num1_ampliado = (int32_t)num1_Q3_12 * 256;
106
107     printf("\n--- Punto 5: Division ---\n");
108     if (num2_Q7_8 == 0) {
109         puts("Error: El divisor es cero en Q7.8.");
110         return 1;
111     }
112
113     int32_t division_Q3_12 = num1_ampliado / num2_Q7_8;
114     mostrar_resultado("Division ajustada a Q3.12", division_Q3_12, 4096);
115
116     return 0;
117 }
```

Figura 4

Figura 4: División en Q3.12 y control de divisor cero (líneas 100 a 117).

Explicación del código en C

1. Conversión de Punto Flotante a Formato Q (Puntos 1 y 2)

Para almacenar los números en la memoria de forma eficiente, partimos de variables float y las convertimos a formatos de punto fijo utilizando enteros de 16 bits (int16_t).

- Q3.12: Multiplicamos el valor original por 4096 (2^12) y aplicamos un casting a int16_t. Esto reserva 3 bits para la parte entera, 12 para la fraccionaria y 1 para el signo.
- Q7.8: Multiplicamos el valor original por 256 (2^8) y aplicamos el casting, reservando 7 bits enteros y 8 fraccionarios.

2. Compatibilización para Suma y Resta (Punto 3)

Parte Crítica: No es posible sumar matemáticamente un número Q3.12 con un Q7.8 de forma directa porque sus "comas virtuales" están desalineadas (tienen distinto peso posicional).

Solución: Se aplica un corrimiento aritmético de cuatro posiciones mediante corrimiento_derecha(). La representación pasa de 12 a 8 bits fraccionarios y conserva el signo. Con ambos operandos en Q7.8 se calculan la suma y la resta.

3. Multiplicación y Ajuste de Formato (Punto 4)

Parte crítica: Al multiplicar Q3.12 por Q7.8, los 12 y 8 bits fraccionarios se suman. El producto crudo tiene 20 bits fraccionarios y se conserva en un entero de 32 bits.

Solución: Se desplaza el producto 12 posiciones a la derecha para expresarlo en Q7.8. Se comprueba que el valor ajustado cabe en 16 bits antes de convertirlo. La suma y la resta se comprueban del mismo modo.

4. División y Conservación de Resolución (Punto 5)

Parte crítica: El cociente directo de los enteros queda escalado por 2^(12 − 8), lo que solo conserva cuatro bits fraccionarios.

Solución: Se multiplica el numerador por 256 antes de dividir. Esto conserva 12 bits fraccionarios en el cociente Q3.12, incluso con numeradores negativos. El programa comprueba el rango del resultado y avisa si el divisor es cero.

2.1.2. Lenguaje Assembly

```
        .ORIG x3000

; Operandos en Q3.12 y Q7.8. Las tres salidas se guardan en Q7.8.
; EST_SUMA, EST_RESTA y EST_MULT: 0 = valido, 1 = fuera de rango.
; Si un estado vale 1, el resultado asociado se guarda en cero.

; Compatibilizar el primer operando con un corrimiento aritmetico de 4.
; Copiar los 12 bits altos, extendiendo el signo en el acumulador R0.
        LD R1, NUM1_Q3_12
        AND R0, R0, #0
        ADD R1, R1, #0
        BRzp INICIO_Q7
        ADD R0, R0, #-1
INICIO_Q7
        AND R2, R2, #0
        ADD R2, R2, #12
Q7_BIT
        ADD R0, R0, R0
        ADD R1, R1, #0
```

```
        BRzp Q7_SIGUIENTE
        ADD R0, R0, #1
Q7_SIGUIENTE
        ADD R1, R1, R1
        ADD R2, R2, #-1
        BRp Q7_BIT

; Suma. Hay desborde si los operandos tienen el mismo signo y el
; resultado cambia de signo.
        LD R1, NUM2_Q7_8
        ADD R2, R0, R1
        AND R3, R3, #0
        ADD R0, R0, #0
        BRn SUMA_A_NEG
        ADD R1, R1, #0
        BRn GUARDA_SUMA
        ADD R2, R2, #0
        BRn SUMA_FUERA
        BRnzp GUARDA_SUMA
SUMA_A_NEG
        ADD R1, R1, #0
        BRzp GUARDA_SUMA
        ADD R2, R2, #0
        BRn GUARDA_SUMA
SUMA_FUERA
        ADD R3, R3, #1
        AND R2, R2, #0
GUARDA_SUMA
        ST R2, RES_SUMA
        ST R3, EST_SUMA

; Resta. Hay desborde si los operandos tienen signos opuestos y el
; resultado cambia respecto del signo del primer operando.
        NOT R2, R1
        ADD R2, R2, #1
        ADD R2, R0, R2
        AND R3, R3, #0
        ADD R0, R0, #0
        BRn RESTA_A_NEG
        ADD R1, R1, #0
        BRzp GUARDA_RESTA
        ADD R2, R2, #0
        BRn RESTA_FUERA
        BRnzp GUARDA_RESTA
RESTA_A_NEG
        ADD R1, R1, #0
        BRn GUARDA_RESTA
        ADD R2, R2, #0
        BRn GUARDA_RESTA
RESTA_FUERA
        ADD R3, R3, #1
        AND R2, R2, #0
GUARDA_RESTA
        ST R2, RES_RESTA
        ST R3, EST_RESTA

; Multiplicar los operandos originales por magnitud. R0 y R1 contienen
; las magnitudes sin signo, incluido x8000 para el extremo negativo.
        AND R7, R7, #0
        LD R0, NUM1_Q3_12
        BRzp MULT_SIGNO_B
        NOT R0, R0
        ADD R0, R0, #1
        ADD R7, R7, #1
MULT_SIGNO_B
        LD R1, NUM2_Q7_8
        BRzp MULT_INICIO
        NOT R1, R1
        ADD R1, R1, #1
```

```
        NOT R7, R7
        ADD R7, R7, #2
MULT_INICIO
        ST R7, SIGNO_PRODUCTO
        AND R2, R2, #0
        AND R3, R3, #0
        AND R4, R4, #0
        LD R5, MASCARA15
        AND R6, R6, #0
        ADD R6, R6, #15
        ADD R6, R6, #1

; Producto crudo Q10.20: R2 + R3*2^15 + R4*2^30.
; Recorrer los 16 bits del multiplicador desde el mas significativo.
; En cada vuelta, duplicar el producto parcial y sumar R0 si el bit es 1.
MULT_BIT
        ADD R4, R4, R4
        ADD R3, R3, R3
        BRzp DOBLA_BAJO
        AND R3, R3, R5
        ADD R4, R4, #1
DOBLA_BAJO
        ADD R2, R2, R2
        BRzp MULT_SUMAR
        AND R2, R2, R5
        ADD R3, R3, #1
MULT_SUMAR
        ADD R1, R1, #0
        BRzp MULT_SIGUIENTE
        ADD R2, R2, R0
        BRzp MULT_SIGUIENTE
        AND R2, R2, R5
        ADD R3, R3, #1
        BRzp MULT_SIGUIENTE
        AND R3, R3, R5
        ADD R4, R4, #1
MULT_SIGUIENTE
        ADD R1, R1, R1
        ADD R6, R6, #-1
        BRp MULT_BIT

; El corrimiento de 12 para Q7.8 admite magnitudes crudas menores
; que 2^27. La magnitud 2^27 exacta solo cabe con signo negativo.
        ADD R4, R4, #0
        BRp MULT_FUERA
        LD R5, NEG_4096
        ADD R5, R3, R5
        BRp MULT_FUERA
        BRn MULT_AJUSTE
        LD R5, SIGNO_PRODUCTO
        BRz MULT_FUERA
        ADD R2, R2, #0
        BRnp MULT_FUERA
        LD R0, MINIMO_Q7
        BRnzp MULT_OK

; Cociente por 4096: ocho veces la palabra media mas los tres
; bits superiores de la palabra baja. Los 12 restantes son el resto.
MULT_AJUSTE
        ADD R0, R3, R3
        ADD R0, R0, R0
        ADD R0, R0, R0
        AND R1, R1, #0
        ADD R2, R2, R2
        AND R6, R6, #0
        ADD R6, R6, #3
AJUSTE_BIT
        ADD R1, R1, R1
        ADD R2, R2, #0
```

```
        BRzp AJUSTE_SIGUIENTE
        ADD R1, R1, #1
AJUSTE_SIGUIENTE
        ADD R2, R2, R2
        ADD R6, R6, #-1
        BRp AJUSTE_BIT
        ADD R0, R0, R1
        LD R5, SIGNO_PRODUCTO
        BRz MULT_OK
        ADD R2, R2, #0
        BRz MULT_NEGAR
        ADD R0, R0, #1
MULT_NEGAR
        NOT R0, R0
        ADD R0, R0, #1
MULT_OK
        AND R1, R1, #0
        BRnzp GUARDA_MULT
MULT_FUERA
        AND R0, R0, #0
        AND R1, R1, #0
        ADD R1, R1, #1
GUARDA_MULT
        ST R0, RES_MULT
        ST R1, EST_MULT
        HALT

NUM1_Q3_12      .FILL #256      ; 0.0625 en Q3.12
NUM2_Q7_8       .FILL #32       ; 0.125 en Q7.8
MASCARA15       .FILL x7FFF
NEG_4096        .FILL #-4096
MINIMO_Q7       .FILL x8000
SIGNO_PRODUCTO  .BLKW #1
RES_SUMA        .BLKW #1
RES_RESTA       .BLKW #1
RES_MULT        .BLKW #1
EST_SUMA        .BLKW #1
EST_RESTA       .BLKW #1
EST_MULT        .BLKW #1
        .END
```

Explicación del Código en Assembly

1. Principio de funcionamiento del algoritmo

El programa en Assembly LC-3 toma dos operandos de memoria. Compatibiliza los formatos, calcula la suma y la resta, y multiplica los valores originales para conservar la precisión del primer operando.

- Compatibilización de formatos: Se copian los 12 bits superiores del primer operando y se extiende el signo para reproducir un corrimiento aritmético de cuatro posiciones. El resultado queda en Q7.8.
- Multiplicación: Se recorre el multiplicador bit por bit y se acumula el producto por magnitud en palabras de 15 bits útiles. Los acarreos preservan el producto crudo. Luego se ajusta la escala de 20 a 8 bits fraccionarios y se aplica el signo, con el mismo truncamiento que en C.

2. Formatos de Punto Fijo

La arquitectura de la LC-3 utiliza registros de 16 bits. Se implementaron dos representaciones en complemento a 2:

- Formato Q3.12: 1 bit de signo, 3 bits para la parte entera y 12 bits para la parte fraccionaria (Factor de escala: 2^12 = 4096).
- Formato Q7.8: 1 bit de signo, 7 bits para la parte entera y 8 bits para la parte fraccionaria (Factor de escala: 2^8 = 256).

3. Rango de valores (Entradas y Salidas)

Rango de valores aceptados (Entrada):

- Operando Q3.12: Desde −8 hasta 7,999755859375. Los extremos se codifican como x8000 y x7FFF.
- Operando Q7.8: Desde −128 hasta 127,99609375. Los extremos se codifican como x8000 y x7FFF.

La suma, la resta y el producto pueden exceder el rango de salida aun con operandos válidos. Assembly indica ese caso con un estado igual a 1 y guarda cero como valor de salida para la operación afectada.

Rango de valores de la salida:

- Suma y resta en Q7.8: Los resultados se leen en RES_SUMA y RES_RESTA. EST_SUMA y EST_RESTA valen 0 para un resultado válido y 1 si hubo desbordamiento.
- Multiplicación en Q7.8: El resultado ajustado se lee en RES_MULT. EST_MULT vale 0 para un resultado válido y 1 si hubo desbordamiento.

4. Precisión alcanzada

La precisión (resolución mínima representable o LSB – Least Significant Bit) depende de la cantidad de bits asignados a la parte fraccionaria en cada formato:

- Precisión en Q3.12: 1 / (2^12) = 1 / 4096 = 0.00024414 (resolución muy alta, ideal para retener decimales finos).
- Precisión en Q7.8: 1 / (2^8) = 1 / 256 = 0.00390625 (resolución media, sacrifica decimales para permitir enteros más grandes).

Prueba reproducible de 3A: con 0,0625 y 0,125, la suma es 0,1875, la resta es −0,0625 y la multiplicación es 0,0078125. La división en C da 0,5. En la LC-3, las tres salidas Q7.8 son x0030, xFFF0 y x0002, con estado 0.

2.2. LABORATORIO 3B

2.2.1. Lenguaje de alto nivel en C

```c
/* Laboratorio 3 - EXP multiplicativa de alta radix (base 16).
 * Compilar: gcc -std=c11 -Wall -Wextra -O2 exp_alta_radix.c -o exp_lab -lm
 * Ejecutar: exp_lab 1.0   |   exp_lab --traza 1.0   |   exp_lab --pruebas
 *
 * Entrada: Q3.12 con signo (int16_t), valor real = entero / 4096.
 * Salida:  Q7.8  con signo (int16_t), valor real = entero / 256.
 * Convencion Q: el bit de signo se cuenta por separado.
 * El calculo usa enteros. exp() solo se usa para validar en el programa PC.
 * Los temporales de 32/64 bits requeriran varias palabras en la LC-3.
 *
 * RECORDATORIO DE PUNTO FIJO:
 *   entero_guardado = redondeo(valor_real * escala)
 *   valor_real      = entero_guardado / escala
 * No hay una coma almacenada en memoria: la escala define su posicion.
 * Ejemplo de entrada: 1.5 se guarda como 6144, porque 6144/4096 = 1.5.
 * Ejemplo de salida:  e^1 se aproxima por 696/256 = 2.71875.
 *
 * Antes de sumar o restar, ambos operandos deben tener la misma escala.
 * La entrada tiene escala 2^12; el nucleo usa escala 2^30 para reducir
 * errores acumulados; la salida se redondea finalmente a escala 2^8.
 * Estos formatos son elecciones de diseno para el ejercicio EXP del PDF.
 */
#include <errno.h>
#include <math.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define ITERACIONES 6
#define UNO_Q30 UINT32_C(1073741824)
#define LN2_Q30 INT64_C(744261118)
#define MAX_X_SIN_SATURAR 19873

/* Rangos de los formatos externos:
 * Q3.12: [-8, 7.999755859375], paso 1/4096.
 * Q7.8:  [-128, 127.99609375], paso 1/256.
 * EXP devuelve solamente resultados no negativos.
 * 19873/4096 es el mayor x Q3.12 cuyo e^x cabe en la salida.
 */
```

Figura 5

Figura 5: Definición de constantes y límite máximo de saturación

```c
typedef enum { EXP_OK, EXP_SATURADO, EXP_REDONDEADO_A_CERO } Estado;
typedef struct {
    int16_t valor;
    Estado estado;
    unsigned iteraciones;
} Resultado;

/* Tabla[j-1][d] = redondeo(ln(1 + d*16^(-j)) * 2^30).
 * Constantes precalculadas: no se necesita log() al ejecutar el algoritmo.
 * Se admite d=16 como digito redundante para asegurar la convergencia.
 */
static const uint32_t tabla_log[ITERACIONES][17] = {
    {0, 65095192, 126468572, 184522808, 239598564, 291986604, 341937090, 3896668
 {0, 4186133, 8356010, 12509755, 16647494, 20769348, 24875440, 28965890, 3304081
    {0, 262112, 524160, 786144, 1048064, 1309921, 1571713, 1833442, 2095107, 235
 {0, 16384, 32768, 49151, 65534, 81917, 98300, 114682, 131064, 147446, 163828, 18
    {0, 1024, 2048, 3072, 4096, 5120, 6144, 7168, 8192, 9216, 10240, 11264, 1228
 {0, 64, 128, 192, 256, 320, 384, 448, 512, 576, 640, 704, 768, 832, 896, 960, 10
};

/* Multiplicar por un digito pequeno usando sumas y desplazamientos. */
static uint64_t multiplicar_digito(uint32_t valor, unsigned digito)
{
    uint64_t suma = 0, termino = valor;
    while (digito != 0) {
        if (digito & 1u) suma += termino;
        termino <<= 1;
        digito >>= 1;
    }
    return suma;
}

/* Rutina que se puede estudiar y luego traducir a assembly.
 * traza=NULL desactiva los mensajes de depuracion.
 * Solo los mensajes de traza convierten temporales a double.
 */
```

Figura 6

Figura 6: Estructura de datos y tabla pre-calculada

```
Resultado exp_q3_12(int16_t entrada, FILE *traza)
{
    Resultado resultado = {0, EXP_OK, 0};
    if (entrada > MAX_X_SIN_SATURAR) {
        resultado.valor = INT16_MAX;
        resultado.estado = EXP_SATURADO;
        return resultado;
    }

    /* x = k*ln(2) + r, con 0 <= r < ln(2).
    * Para pasar de escala 2^12 a 2^30, multiplicamos por 2^(30-12).
    * El valor representado no cambia, solo cambia el entero que lo guarda.
    * Multiplicar evita el desplazamiento izquierdo de negativos,
    * que tiene comportamiento indefinido en C11.
    * La division de C trunca hacia cero: se corrige para obtener piso.
    */
    int64_t x_q30 = (int64_t)entrada * INT64_C(262144);
    int k = (int)(x_q30 / LN2_Q30);
    int64_t residuo = x_q30 - (int64_t)k * LN2_Q30;
    if (residuo < 0) {
        --k;
        residuo += LN2_Q30;
    }
    uint32_t acumulador = UNO_Q30; /* E=1, escala interna 2^30. */
    if (traza) {
        fprintf(traza, "Reduccion: k=%d, r=%.10f\n", k,
                (double)residuo / UNO_Q30);
        fprintf(traza, "Cada entero interno se interpreta dividiendolo por 2^30.\n");
        fprintf(traza, "Paso  Digito  Residuo entero  Residuo real     E entero     E real\n");
    }

    for (unsigned j = 1; j <= ITERACIONES; ++j) {
        /* Mayor d tal que ln(1+d*16^-j) <= residuo.
        * La tabla y el residuo tienen escala 2^30: podemos compararlos
        * y restarlos directamente sin desalinear la coma.
        */
        unsigned d = 16;
        while (d > 0 && (int64_t)tabla_log[j - 1][d] > residuo) --d;
        residuo -= tabla_log[j - 1][d];
```

Figura 7

Figura 7: Inicio de rutina exponencial y reducción de rango

```
        /* E <- E*(1+d*16^-j) = E + (E*d)/2^(4j).
        * Se redondea al entero mas cercano en la escala interna.
        * d es un entero sin parte fraccionaria: E*d conserva escala 2^30.
        * Dividir por 2^(4j) aplica el factor 16^-j del algoritmo.
        * Se suma medio divisor antes de desplazar para redondear.
        * El temporal de 64 bits evita perder los bits altos del producto.
        */
        unsigned bits = 4u * j;
        uint64_t producto = multiplicar_digito(acumulador, d);
        acumulador += (uint32_t)((producto + (UINT64_C(1) << (bits - 1))) >> bits);
        ++resultado.iteraciones;
        if (traza) fprintf(traza, "%4u  %6u  %14lld  %.10f  %10llu  %.10f\n", j, d,
                           (long long)residuo, (double)residuo / UNO_Q30,
                           (unsigned long long)acumulador, residuo: int64_t or / UNO_Q30);
    }

    /* e^x = E*2^k. Pasar de 30 a 8 bits fraccionarios:
    * salida_entera = redondeo(E_entero / 2^(22-k)).
    * Los 22 bits provienen de 30-8. Se incorpora k en el mismo ajuste
    * para aplicar 2^k y redondear una sola vez a la escala de salida.
    * En el dominio admitido, k pertenece a [-12,6], bits a [16,34].
    */
    unsigned bits = (unsigned)(22 - k);
    uint64_t salida = ((uint64_t)acumulador + (UINT64_C(1) << (bits - 1))) >> bits;
    if (salida > INT16_MAX) {
        resultado.valor = INT16_MAX;
        resultado.estado = EXP_SATURADO;
    } else {
        resultado.valor = (int16_t)salida;
        if (salida == 0) resultado.estado = EXP_REDONDEADO_A_CERO;
    }
    return resultado;
}

static const char *nombre_estado(Estado estado)
{
    if (estado == EXP_SATURADO) return "saturado: excede Q7.8";
    if (estado == EXP_REDONDEADO_A_CERO) return "redondeado a cero: menor que medio paso Q7.8";
    return "OK";
}
```

Figura 8

Figura 8: Núcleo iterativo multiplicativo y ajuste de salida

```c
/* Barrido exhaustivo de las 65536 entradas Q3.12.
 * El error se mide contra exp(x cuantizado), excluyendo saturacion.
 */
static int pruebas(void)
{
    double max_abs = 0, min_abs = HUGE_VAL, max_rel = 0, peor_x = 0;
    unsigned saturados = 0, ceros = 0, diferencias = 0, fallos = 0;
    int16_t anterior = 0;
    for (int32_t n = INT16_MIN; n <= INT16_MAX; ++n) {
        Resultado r = exp_q3_12((int16_t)n, NULL);
        double x = (double)n / 4096.0, referencia = exp(x);
        if (n > INT16_MIN && r.valor < anterior) ++fallos;
        anterior = r.valor;
        if (r.estado == EXP_SATURADO) {
            ++saturados;
            if (n <= MAX_X_SIN_SATURAR || r.valor != INT16_MAX || r.iteraciones != 0) ++fallos;
            continue;
        }
        if (n > MAX_X_SIN_SATURAR || r.iteraciones != ITERACIONES) ++fallos;
        double error = fabs((double)r.valor / 256.0 - referencia);
        if (error < min_abs) min_abs = error;
        if (error > max_abs) { max_abs = error; peor_x = x; }
        if (error / referencia > max_rel) max_rel = error / referencia;
        if (r.estado == EXP_REDONDEADO_A_CERO) ++ceros;
        if ((long)r.valor != lround(referencia * 256.0)) ++diferencias;
        if (error > 1.0 / 256.0) ++fallos;
    }
    if (exp_q3_12(0, NULL).valor != 256) ++fallos;
    printf("Entradas verificadas: 65536\nSaturadas: %u\nRedondeadas a cero: %u\n", saturados, ceros);
    printf("Error absoluto minimo (sin saturacion): %.12g\n", min_abs);
    printf("Error absoluto maximo (sin saturacion): %.12g en x=%.12f\n", max_abs, peor_x);
    printf("Error relativo maximo (sin saturacion): %.6f %%\n", max_rel * 100);
    printf("Diferencias frente al redondeo ideal Q7.8: %u\n", diferencias);
    printf("Fallos (monotonia, estados, iteraciones o error > 1 LSB): %u\n", fallos);
    return fallos ? EXIT_FAILURE : EXIT_SUCCESS;
}
```

Figura 9

Figura 9: Función de validación y barrido de valores posibles del formato de 16 bits

```c
int main(int argc, char **argv){
    char linea[128], *fin;
    const char *texto;
    FILE *traza = NULL;
    if (argc == 2 && strcmp(argv[1], "--pruebas") == 0) return pruebas();
    if (argc == 3 && strcmp(argv[1], "--traza") == 0) {
        texto = argv[2];
        traza = stdout;
    } else if (argc == 2) {
        texto = argv[1];
    } else if (argc == 1) {
        printf("Ingrese x en [-8, 7.999755859375], usando punto decimal: ");
        if (!fgets(linea, sizeof linea, stdin)) return EXIT_FAILURE;
        texto = linea;
    } else {
        fprintf(stderr, "Uso: %s [x | --traza x | --pruebas]\n", argv[0]);
        return EXIT_FAILURE;
    }
    errno = 0;
    double x = strtod(texto, &fin);
    if (fin == texto) {
        fprintf(stderr, "Entrada invalida. Use un numero con punto decimal.\n");
        return EXIT_FAILURE;
    }
    while (*fin == ' ' || *fin == '\t' || *fin == '\r' || *fin == '\n') ++fin;
    if (errno == ERANGE || *fin || !isfinite(x) || x < -8 || x > 32767.0 / 4096.0) {
        fprintf(stderr, "Entrada fuera de Q3.12 o invalida.\n");
        return EXIT_FAILURE;
    }
    /* Frontera de entrada: del decimal ingresado al entero Q3.12.
     * lround redondea al entero mas cercano, con empates alejandose de cero.
     * El rango ya se comprobo antes de convertir a int16_t.
     */
    int16_t entrada = (int16_t)lround(x * 4096.0);
    double x_q = (double)entrada / 4096.0;
    Resultado r = exp_q3_12(entrada, traza);
    printf("x solicitado: %.12f\nx Q3.12: %d -> %.12f\n", x, (int)entrada, x_q);
    /* Frontera de salida: dividir por 256 permite mostrar el valor real.
     * Esta conversion a double no forma parte del calculo de EXP.
     */
    printf("Resultado Q7.8: %d -> %.12f\n", (int)r.valor, (double)r.valor / 256.0);
    printf("Referencia exp(x Q3.12): %.12f\n", exp(x_q));
    printf("Error absoluto respecto de x Q3.12: %.12f\n", fabs((double)r.valor / 256.0 - exp(x_q)));
    printf("Iteraciones: %u\nEstado: %s\n", r.iteraciones, nombre_estado(r.estado));
    return EXIT_SUCCESS;
}
```

Figura 10

Figura 10: Función principal e interfaz de usuario

2.2.2. Lenguaje Assembly

```
; EXP para LC-3. Entrada Q3.12 y salida Q7.8 de 16 bits con signo.
; Ensamblar, cargar y ejecutar desde x3000. Cambiar ENTRADA para probar.
; Ejemplo: x1000 = 1.0 -> RES_EXP = x02B8 = 696 -> 696/256 = 2.71875.
; EST_EXP: 0 = normal, 1 = saturado, 2 = redondeado a cero.
; NUM_ITER: 6 en el calculo normal, 0 si se detecta saturacion al entrar.
;
; EXP_Q3_12 es invocable con JSR. R0 entra como Q3.12 y retorna Q7.8.
; R1 retorna el estado y R2, las iteraciones. Modifica R3-R6 y CC.
; Usa memoria de trabajo estatica. R7 se guarda antes de llamar auxiliares.
; La entrada y la salida se pasan como enteros ya codificados en punto fijo.
;
; Representacion interna: escala 2^30, en tres palabras de 15 bits utiles.
; Un entero V se representa como V0 + V1*2^15 + V2*2^30.
; Se reserva el bit alto de cada palabra para detectar acarreos/prestamos.
;
; Bloque de trabajo (R6 como base):
;  0  retorno de EXP          1-3  residuo (bajo, medio, alto)
;  4  k                       5    bandera de entrada negativa
;  6-8 acumulador E           9    reservado
; 10  puntero a fila de tabla 11   desplazamiento 4*j
; 12  iteraciones             13-15 temporal para desplazamiento
; 16  retorno de REDONDEA     17   puntero de lectura de REDONDEA

        .ORIG x3000
        LD R0, ENTRADA
        JSR EXP_Q3_12
        ST R0, RES_EXP
        ST R1, EST_EXP
        ST R2, NUM_ITER
        HALT

ENTRADA .FILL x1000
```

```
RES_EXP .BLKW #1
EST_EXP .BLKW #1
NUM_ITER .BLKW #1

EXP_Q3_12
        LD R6, PTR_TRABAJO
        STR R7, R6, #0
        AND R1, R1, #0
        STR R1, R6, #4
        STR R1, R6, #5
        STR R1, R6, #12
        ADD R0, R0, #0
        BRn EXP_NEG
        LD R2, NEG_LIMITE
        ADD R2, R0, R2
        BRp EXP_SAT
        BRnzp ESCALA
EXP_NEG
        ADD R1, R1, #1
        STR R1, R6, #5
        NOT R0, R0
        ADD R0, R0, #1      ; Magnitud sin signo, incluso x8000.

ESCALA
        ; De Q3.12 a escala 2^30: magnitud << 18.
        ; V0=0, V1=(magnitud << 3) mod 2^15, V2=bits 15..12.
        AND R1, R1, #0
        STR R1, R6, #1
        ADD R1, R0, R0
        ADD R1, R1, R1
        ADD R1, R1, R1
        LD R5, MASK15
        AND R1, R1, R5
        STR R1, R6, #2
```

```
        AND R2, R2, #0
        AND R3, R3, #0
        ADD R3, R3, #4
ESCALA_BITS
        ADD R2, R2, R2
        ADD R0, R0, #0
        BRzp ESCALA_SIG
        ADD R2, R2, #1
ESCALA_SIG
        ADD R0, R0, R0
        ADD R3, R3, #-1
        BRp ESCALA_BITS
        STR R2, R6, #3

        ; Reducir |x| por restas de ln(2), sin division de hardware.
        ; Si la resta de prueba es negativa se conserva el residuo anterior.
REDUCE
        LDR R0, R6, #1
        LDR R1, R6, #2
        LDR R2, R6, #3
        LD R3, NEG_LN_LO
        ADD R0, R0, R3
        BRzp REDUCE_MED
        ADD R1, R1, #-1
REDUCE_MED
        LD R3, NEG_LN_MI
        ADD R1, R1, R3
        BRzp REDUCE_ALTO
        ADD R2, R2, #-1
REDUCE_ALTO
        ADD R2, R2, #0
        BRn REDUCE_SIGNO
        AND R0, R0, R5
        AND R1, R1, R5
```

```
        STR R0, R6, #1
        STR R1, R6, #2
        STR R2, R6, #3
        LDR R3, R6, #4
        ADD R3, R3, #1
        STR R3, R6, #4
        BRnzp REDUCE

REDUCE_SIGNO
        LDR R0, R6, #5
        BRz EXP_INICIA
        LDR R2, R6, #4
        NOT R2, R2          ; Si hay resto, k = -cociente - 1.
        LDR R0, R6, #1
        LDR R1, R6, #2
        ADD R3, R0, R1      ; Suma cero solo si ambas palabras son cero.
        BRnp REDUCE_COMP
        ADD R2, R2, #1      ; Resto cero: k = -cociente.
        BRnzp REDUCE_GUARDA
REDUCE_COMP
        ; Residuo de x negativo: ln(2) - residuo de |x|.
        NOT R0, R0
        ADD R0, R0, #1
        NOT R1, R1
        ADD R1, R1, #1
        LD R3, LN_LO
        ADD R0, R0, R3
        BRzp REDUCE_COMP_MI
        ADD R1, R1, #-1
REDUCE_COMP_MI
        LD R3, LN_MI
        ADD R1, R1, R3
        AND R0, R0, R5
        STR R0, R6, #1
```

```
        STR R1, R6, #2
REDUCE_GUARDA
        STR R2, R6, #4

EXP_INICIA
        ; E = 1 -> entero 2^30 -> palabras [0, 0, 1].
        AND R0, R0, #0
        STR R0, R6, #6
        STR R0, R6, #7
        ADD R0, R0, #1
        STR R0, R6, #8
        ADD R0, R0, #3
        STR R0, R6, #11
        LD R0, PTR_D16
        STR R0, R6, #10
        BRnzp EXP_PASO

; Constantes de reduccion. ln(2)*2^30 redondeado = 744261118.
PTR_TRABAJO .FILL TRABAJO
NEG_LIMITE  .FILL #-19873
MASK15      .FILL x7FFF
PTR_D16     .FILL TABLA_D16
LN_LO       .FILL #1534
LN_MI       .FILL #22713
NEG_LN_LO   .FILL #-1534
NEG_LN_MI   .FILL #-22713

EXP_PASO
        LDR R3, R6, #10    ; Comenzar por el digito 16 de la fila.
        AND R5, R5, #0
        ADD R5, R5, #15
        ADD R5, R5, #1
EXP_BUSCA
        LDR R0, R6, #1
```

```
        LDR R1, R6, #2
        LDR R4, R3, #0     ; Probar r - tabla[j][d] en dos palabras.
        NOT R4, R4
        ADD R4, R4, #1
        ADD R0, R0, R4
        BRzp EXP_BUSCA_MI
        ADD R1, R1, #-1
EXP_BUSCA_MI
        LDR R4, R3, #1
        NOT R4, R4
        ADD R4, R4, #1
        ADD R1, R1, R4
        BRzp EXP_ACEPTA
        ADD R3, R3, #-2    ; Digito anterior en la misma fila.
        ADD R5, R5, #-1
        BRnzp EXP_BUSCA
EXP_ACEPTA
        ADD R3, R5, #0     ; Guardar digito en R3 para el producto.
        LD R5, MASK15
        AND R0, R0, R5
        STR R0, R6, #1
        STR R1, R6, #2

        ; Producto E*d por sumas. d <= 16. R2:R1:R0 acumula.
        AND R0, R0, #0
        AND R1, R1, #0
        AND R2, R2, #0
        ADD R3, R3, #0
        BRz EXP_MULT_FIN
EXP_MULT
        LDR R4, R6, #6
        ADD R0, R0, R4
        BRzp EXP_MULT_MI
        AND R0, R0, R5
```

```
        ADD R1, R1, #1
EXP_MULT_MI
        LDR R4, R6, #7
        ADD R1, R1, R4
        BRzp EXP_MULT_ALTO
        AND R1, R1, R5
        ADD R2, R2, #1
EXP_MULT_ALTO
        LDR R4, R6, #8
        ADD R2, R2, R4
        ADD R3, R3, #-1
        BRp EXP_MULT
EXP_MULT_FIN
        STR R0, R6, #13
        STR R1, R6, #14
        STR R2, R6, #15
        LDR R0, R6, #11    ; Dividir E*d por 2^(4*j) con redondeo.
        JSR REDONDEA

        ; Acumular E <- E + redondeo(E*d/2^(4*j)).
        LD R5, MASK15
        LDR R3, R6, #6
        ADD R0, R0, R3
        BRzp EXP_SUMA_MI
        AND R0, R0, R5
        ADD R1, R1, #1
EXP_SUMA_MI
        LDR R3, R6, #7
        ADD R1, R1, R3
        BRzp EXP_SUMA_ALTO
        AND R1, R1, R5
        ADD R2, R2, #1
EXP_SUMA_ALTO
        LDR R3, R6, #8
```

```
        ADD R2, R2, R3
        STR R0, R6, #6
        STR R1, R6, #7
        STR R2, R6, #8
        LDR R3, R6, #12
        ADD R3, R3, #1
        STR R3, R6, #12
        ADD R3, R3, #-6
        BRz EXP_SALIDA
        LDR R0, R6, #10
        ADD R0, R0, #15
        ADD R0, R0, #15
        ADD R0, R0, #4     ; 17 constantes de dos palabras por fila.
        STR R0, R6, #10
        LDR R0, R6, #11
        ADD R0, R0, #4
        STR R0, R6, #11
        BRnzp EXP_PASO

EXP_SALIDA
        ; Q7.8 = redondeo(E_entero / 2^(30-8-k)).
        STR R0, R6, #13
        STR R1, R6, #14
        STR R2, R6, #15
        LDR R0, R6, #4
        NOT R0, R0
        ADD R0, R0, #1
        ADD R0, R0, #15
        ADD R0, R0, #7
        JSR REDONDEA
        ADD R3, R1, R2    ; Palabras altas distintas de cero: no cabe.
        BRnp EXP_SAT
        ADD R0, R0, #0
        BRnp EXP_FIN      ; R1 ya contiene cero (estado normal).
```

```
        ADD R1, R1, #2
        BRnzp EXP_FIN
EXP_SAT
        LD R0, MASK15
        AND R1, R1, #0
        ADD R1, R1, #1
EXP_FIN
        LDR R2, R6, #12
        LDR R7, R6, #0
        RET

; REDONDEA: entero sin signo de 45 bits en TRABAJO+13..15.
; R0 entra con n (1..44). Retorna V/2^n redondeado en R2:R1:R0.
; Lee los 45-n bits altos y el siguiente bit decide el redondeo.
; En los usos de EXP, el resultado cabe en las tres palabras.
REDONDEA
        STR R7, R6, #16
        LD R7, MASK15
        NOT R5, R0
        ADD R5, R5, #1
        ADD R5, R5, #15
        ADD R5, R5, #15
        ADD R5, R5, #15    ; 45-n: bits que hay que copiar.
        AND R0, R0, #0
        AND R1, R1, #0
        AND R2, R2, #0
        ADD R3, R6, #15
RED_CARGA
        STR R3, R6, #17
        LDR R3, R3, #0
        ADD R3, R3, R3     ; Bit 14 de la palabra pasa al signo.
        AND R4, R4, #0
        ADD R4, R4, #15
RED_BIT
```

```
        ADD R5, R5, #0
        BRz RED_AJUSTA
        ADD R2, R2, R2
        ADD R1, R1, R1
        BRzp RED_DUP_BAJO
        AND R1, R1, R7
        ADD R2, R2, #1
RED_DUP_BAJO
        ADD R0, R0, R0
        BRzp RED_INSERTA
        AND R0, R0, R7
        ADD R1, R1, #1
RED_INSERTA
        ADD R3, R3, #0
        BRzp RED_SIG_BIT
        ADD R0, R0, #1
RED_SIG_BIT
        ADD R3, R3, R3
        ADD R5, R5, #-1
        ADD R4, R4, #-1
        BRp RED_BIT
        LDR R3, R6, #17
        ADD R3, R3, #-1
        BRnzp RED_CARGA
RED_AJUSTA
        ADD R3, R3, #0
        BRzp RED_RETORNO
        ADD R0, R0, #1
        BRzp RED_RETORNO
        AND R0, R0, R7
        ADD R1, R1, #1
        BRzp RED_RETORNO
        AND R1, R1, R7
        ADD R2, R2, #1
```

```
RED_RETORNO
        LDR R7, R6, #16
        RET

TRABAJO .BLKW #18

; Misma tabla del C. Cada constante se guarda como [bajo15, medio15].
; Las filas se guardan contiguas y el seleccionador empieza en d=16.
; j = 1
        .FILL x0000 ; d=0, bajo
        .FILL x0000 ; medio
        .FILL x4618 ; d=1, bajo
        .FILL x07C2 ; medio
        .FILL x41DC ; d=2, bajo
        .FILL x0F13 ; medio
        .FILL x1838 ; d=3, bajo
        .FILL x15FF ; medio
        .FILL x7BE4 ; d=4, bajo
        .FILL x1C8F ; medio
        .FILL x5CAC ; d=5, bajo
        .FILL x22CE ; medio
        .FILL x0BC2 ; d=6, bajo
        .FILL x28C3 ; medio
        .FILL x57F7 ; d=7, bajo
        .FILL x2E73 ; medio
        .FILL x23ED ; d=8, bajo
        .FILL x33E6 ; medio
        .FILL x77C8 ; d=9, bajo
        .FILL x391F ; medio
        .FILL x0F60 ; d=10, bajo
        .FILL x3E25 ; medio
        .FILL x65C8 ; d=11, bajo
        .FILL x42F9 ; medio
        .FILL x3EBD ; d=12, bajo
```

```
        .FILL x47A1 ; medio
        .FILL x2E67 ; d=13, bajo
        .FILL x4C1F ; medio
        .FILL x1FD1 ; d=14, bajo
        .FILL x5076 ; medio
        .FILL x5A4D ; d=15, bajo
        .FILL x54A8 ; medio
TABLA_D16 .FILL x05FE ; d=16, bajo
        .FILL x58B9 ; medio
; j = 2
        .FILL x0000 ; d=0, bajo
        .FILL x0000 ; medio
        .FILL x6015 ; d=1, bajo
        .FILL x007F ; medio
        .FILL x00AA ; d=2, bajo
        .FILL x00FF ; medio
        .FILL x623B ; d=3, bajo
        .FILL x017D ; medio
        .FILL x0546 ; d=4, bajo
        .FILL x01FC ; medio
        .FILL x6A44 ; d=5, bajo
        .FILL x0279 ; medio
        .FILL x11B0 ; d=6, bajo
        .FILL x02F7 ; medio
        .FILL x7C02 ; d=7, bajo
        .FILL x0373 ; medio
        .FILL x29B1 ; d=8, bajo
        .FILL x03F0 ; medio
        .FILL x1B31 ; d=9, bajo
        .FILL x046C ; medio
        .FILL x50F7 ; d=10, bajo
        .FILL x04E7 ; medio
        .FILL x4B76 ; d=11, bajo
        .FILL x0562 ; medio
```

```
        .FILL x0B1F ; d=12, bajo
        .FILL x05DD ; medio
        .FILL x1062 ; d=13, bajo
        .FILL x0657 ; medio
        .FILL x5BAE ; d=14, bajo
        .FILL x06D0 ; medio
        .FILL x6D71 ; d=15, bajo
        .FILL x0749 ; medio
        .FILL x4618 ; d=16, bajo
        .FILL x07C2 ; medio
; j = 3
        .FILL x0000 ; d=0, bajo
        .FILL x0000 ; medio
        .FILL x7FE0 ; d=1, bajo
        .FILL x0007 ; medio
        .FILL x7F80 ; d=2, bajo
        .FILL x000F ; medio
        .FILL x7EE0 ; d=3, bajo
        .FILL x0017 ; medio
        .FILL x7E00 ; d=4, bajo
        .FILL x001F ; medio
        .FILL x7CE1 ; d=5, bajo
        .FILL x0027 ; medio
        .FILL x7B81 ; d=6, bajo
        .FILL x002F ; medio
        .FILL x79E2 ; d=7, bajo
        .FILL x0037 ; medio
        .FILL x7803 ; d=8, bajo
        .FILL x003F ; medio
        .FILL x75E4 ; d=9, bajo
        .FILL x0047 ; medio
        .FILL x7385 ; d=10, bajo
        .FILL x004F ; medio
        .FILL x70E7 ; d=11, bajo
```

```
        .FILL x0057 ; medio
        .FILL x6E09 ; d=12, bajo
        .FILL x005F ; medio
        .FILL x6AEB ; d=13, bajo
        .FILL x0067 ; medio
        .FILL x678E ; d=14, bajo
        .FILL x006F ; medio
        .FILL x63F2 ; d=15, bajo
        .FILL x0077 ; medio
        .FILL x6015 ; d=16, bajo
        .FILL x007F ; medio
; j = 4
        .FILL x0000 ; d=0, bajo
        .FILL x0000 ; medio
        .FILL x4000 ; d=1, bajo
        .FILL x0000 ; medio
        .FILL x0000 ; d=2, bajo
        .FILL x0001 ; medio
        .FILL x3FFF ; d=3, bajo
        .FILL x0001 ; medio
        .FILL x7FFE ; d=4, bajo
        .FILL x0001 ; medio
        .FILL x3FFD ; d=5, bajo
        .FILL x0002 ; medio
        .FILL x7FFC ; d=6, bajo
        .FILL x0002 ; medio
        .FILL x3FFA ; d=7, bajo
        .FILL x0003 ; medio
        .FILL x7FF8 ; d=8, bajo
        .FILL x0003 ; medio
        .FILL x3FF6 ; d=9, bajo
        .FILL x0004 ; medio
        .FILL x7FF4 ; d=10, bajo
        .FILL x0004 ; medio
```

```
        .FILL x3FF1 ; d=11, bajo
        .FILL x0005 ; medio
        .FILL x7FEE ; d=12, bajo
        .FILL x0005 ; medio
        .FILL x3FEB ; d=13, bajo
        .FILL x0006 ; medio
        .FILL x7FE8 ; d=14, bajo
        .FILL x0006 ; medio
        .FILL x3FE4 ; d=15, bajo
        .FILL x0007 ; medio
        .FILL x7FE0 ; d=16, bajo
        .FILL x0007 ; medio
; j = 5
        .FILL x0000 ; d=0, bajo
        .FILL x0000 ; medio
        .FILL x0400 ; d=1, bajo
        .FILL x0000 ; medio
        .FILL x0800 ; d=2, bajo
        .FILL x0000 ; medio
        .FILL x0C00 ; d=3, bajo
        .FILL x0000 ; medio
        .FILL x1000 ; d=4, bajo
        .FILL x0000 ; medio
        .FILL x1400 ; d=5, bajo
        .FILL x0000 ; medio
        .FILL x1800 ; d=6, bajo
        .FILL x0000 ; medio
        .FILL x1C00 ; d=7, bajo
        .FILL x0000 ; medio
        .FILL x2000 ; d=8, bajo
        .FILL x0000 ; medio
        .FILL x2400 ; d=9, bajo
        .FILL x0000 ; medio
        .FILL x2800 ; d=10, bajo
```

```
        .FILL x0000 ; medio
        .FILL x2C00 ; d=11, bajo
        .FILL x0000 ; medio
        .FILL x3000 ; d=12, bajo
        .FILL x0000 ; medio
        .FILL x3400 ; d=13, bajo
        .FILL x0000 ; medio
        .FILL x3800 ; d=14, bajo
        .FILL x0000 ; medio
        .FILL x3C00 ; d=15, bajo
        .FILL x0000 ; medio
        .FILL x4000 ; d=16, bajo
        .FILL x0000 ; medio
; j = 6
        .FILL x0000 ; d=0, bajo
        .FILL x0000 ; medio
        .FILL x0040 ; d=1, bajo
        .FILL x0000 ; medio
        .FILL x0080 ; d=2, bajo
        .FILL x0000 ; medio
        .FILL x00C0 ; d=3, bajo
        .FILL x0000 ; medio
        .FILL x0100 ; d=4, bajo
        .FILL x0000 ; medio
        .FILL x0140 ; d=5, bajo
        .FILL x0000 ; medio
        .FILL x0180 ; d=6, bajo
        .FILL x0000 ; medio
        .FILL x01C0 ; d=7, bajo
        .FILL x0000 ; medio
        .FILL x0200 ; d=8, bajo
        .FILL x0000 ; medio
        .FILL x0240 ; d=9, bajo
        .FILL x0000 ; medio
```

```
.FILL x0280 ; d=10, bajo
.FILL x0000 ; medio
.FILL x02C0 ; d=11, bajo
.FILL x0000 ; medio
.FILL x0300 ; d=12, bajo
.FILL x0000 ; medio
.FILL x0340 ; d=13, bajo
.FILL x0000 ; medio
.FILL x0380 ; d=14, bajo
.FILL x0000 ; medio
.FILL x03C0 ; d=15, bajo
.FILL x0000 ; medio
.FILL x0400 ; d=16, bajo
.FILL x0000 ; medio
.END
```

Explicación del Código en Assembly

1. Principio de funcionamiento del algoritmo

El algoritmo implementa la función exponencial (e^x) utilizando un método multiplicativo de alta radix (base 16). Para sortear las limitaciones de hardware de la LC-3 y mantener la precisión, el cálculo se divide en las siguientes etapas:

- Reducción de rango: La entrada x se descompone en x = k * ln(2) + r, donde el residuo r se mantiene en el rango 0 <= r < ln(2).
- Iteración multiplicativa: Se utiliza una tabla precalculada de logaritmos. En cada una de las seis iteraciones se elige un dígito d entre 0 y 16. El acumulador E se multiplica por (1 + d · 16^(−j)) mediante sumas y desplazamientos.
- Ajuste final: El resultado acumulado E se multiplica por 2^k y se ajusta el formato desplazando los bits para obtener la salida final redondeada.

2. Formato de Punto Fijo

Se utilizan distintos formatos y escalas para maximizar la resolución durante el cálculo:

- Entrada (Q3.12): 1 bit de signo, 3 bits enteros y 12 fraccionarios. Factor de escala: 2^12 = 4096.
- Escala interna 2^30: La LC-3 representa los valores internos en varias palabras de 16 bits. Para el acumulador y otros temporales se usan hasta tres palabras de 15 bits útiles. El bit superior se utiliza para detectar acarreos.
- Salida (Q7.8): 1 bit de signo, 7 bits enteros y 8 fraccionarios. Factor de escala: 2^8 = 256.

3. Rango de valores (Entradas y Salidas)

Rango de entrada (Q3.12):

- Límite teórico del formato: [-8.0, 7.99975].
- Límite sin saturación: El mayor código de entrada cuyo resultado cabe en Q7.8 es 19873, que representa x = 4,851806640625. Para entradas mayores se devuelve x7FFF y se indica saturación.

Rango de salida (Q7.8):

- Por definición matemática, la función exponencial no devuelve valores negativos. El rango de salida efectivo va desde 0 hasta 127.996 (valor máximo del Q7.8 positivo representado como x7FFF).

4. Precisión alcanzada

La resolución es la diferencia entre dos valores representables consecutivos, también llamada un paso o LSB.

- Precisión de entrada (Q3.12): 1 / 4096 = 0.00024414.
- Precisión de salida (Q7.8): 1 / 256 = 0.00390625.
- El barrido de las 65.536 entradas Q3.12 mostró coincidencia entre C y LC-3 en el resultado, el estado y la cantidad de iteraciones. Frente a exp() de la biblioteca de C, el error absoluto mínimo fue 0 y el máximo fue 0,0019580065, menor que 1/256. Esta medición excluye las entradas saturadas y toma como referencia la entrada ya cuantizada. Las salidas inferiores a medio paso de Q7.8 se redondean a cero.

Prueba reproducible: con ENTRADA en x1000 (x = 1), ejecutar desde x3000 y consultar RES_EXP en x3007. El valor esperado es x02B8, que en Q7.8 representa 2,71875. El estado en x3008 es 0 y el número de iteraciones en x3009 es 6.

3. BITÁCORA DE DESARROLLO

Fecha: 7 y 14 de octubre de 2026
Avances del Lab 3a: Implementación de punto fijo y Assembly

Arrancamos con el Lab 3A. Repasamos el material de la cátedra sobre punto fijo y practicamos operaciones con varios formatos Q. Después armamos el código en Assembly y lo fuimos depurando con pruebas.

Fecha: 21 y 28 de octubre de 2026
Cálculo de exp(e^x) en LC-3: Algoritmo multiplicativo de base 16

Nos tocó implementar el cálculo de la función exp(e^x) aplicando el algoritmo multiplicativo de base 16 con reducción de argumentos. A partir de ahí, nos pusimos a investigar esta aproximación matemática para entender cómo traducirla a Assembly y correrla en el procesador LC-3. El mayor desafío fue ajustar los cálculos a las limitaciones de la arquitectura, arreglándonos con las operaciones disponibles y el formato de punto fijo.

Fecha: 4 de noviembre de 2026
Aprobación y organización

Consultamos con el ayudante de cátedra sobre las implementaciones y los avances. Después trabajamos en la finalización del proyecto y en la elaboración del informe. Organizamos el trabajo colaborativo con Git y GitHub.

Fecha: 5 de noviembre de 2026
Presentación del LABORATORIO 3

Preparamos en GitHub el informe y los códigos en C y Assembly de las dos partes del Laboratorio 3. También subimos al ensamblador del Grupo 8 el código en Assembly de la función EXP, basado en el algoritmo multiplicativo de alta radix

4. CONCLUSIONES

En 3A compatibilizamos Q3.12 y Q7.8 para sumar y restar, y conservamos el producto crudo hasta ajustar la escala. La revisión mostró que el producto podía exceder un registro de la LC-3 incluso si el resultado final cabía en Q7.8. La implementación corregida usa varias palabras y avisa cuando un resultado queda fuera del rango de salida.

Para EXP, el método multiplicativo de base 16 con reducción de argumento completó seis iteraciones para las entradas sin saturación. C y Assembly coincidieron en las 65.536 entradas Q3.12. El error absoluto máximo de las salidas sin saturación fue aproximadamente 0,00195801, por debajo de un paso de Q7.8.